N+1 Problem in Hibernate / Spring Boot

1. What is N+1 Problem?

The **N+1 problem** is a performance issue in ORM frameworks like Hibernate where:

- **1 query** is executed to fetch parent records
- **N additional queries** are executed to fetch child records (one per parent)

So total queries = **N + 1**

This leads to unnecessary database roundtrips and degraded performance.

Simple Example

- 3 Customers
- Each has Addresses
- One customer mapped to Many Customers.
  Let's say we need to get all customers with its addresses.

Instead of:

- 1 query → fetch customers
- 3 queries → fetch addresses

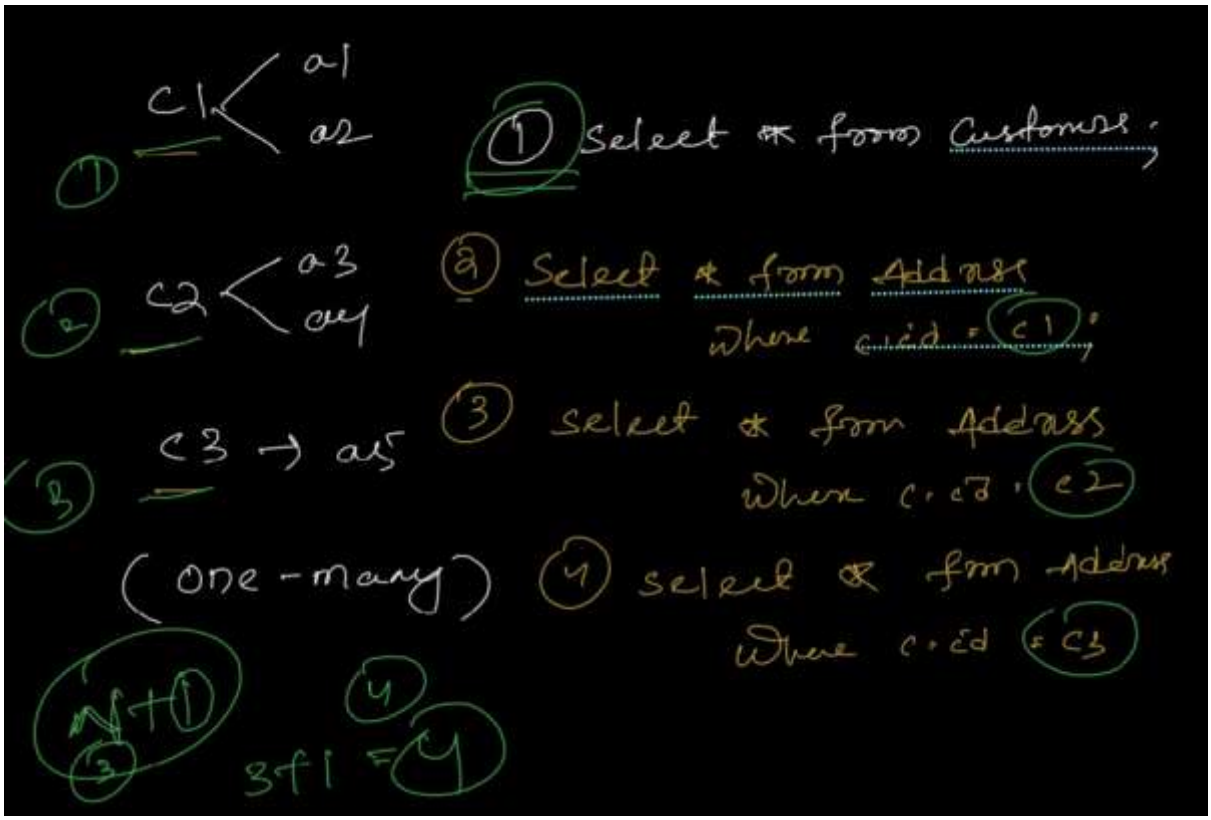
Total = **4 queries (N+1)** → Performance issue

Visual Understanding

2. Why is it called N+1?

- **1 query** → Fetch parent (Department)
- **N queries** → Fetch child (Employees per department)

Hence: **N (children queries) + 1 (parent query)**



In real time, hitting DB multiple times, will downgrade the application performance.


```
Hibernate: select c1_0.id,c1_0.name from Customer c1_0
Hibernate: select a1_0.customer_id,a1_0.id,a1_0.city,a1_0.state,a1_0.zipCode from Address a1_0 whe
Hibernate: select a1_0.customer_id,a1_0.id,a1_0.city,a1_0.state,a1_0.zipCode from Address a1_0 whe
Hibernate: select a1_0.customer_id,a1_0.id,a1_0.city,a1_0.state,a1_0.zipCode from Address a1_0 whe
```

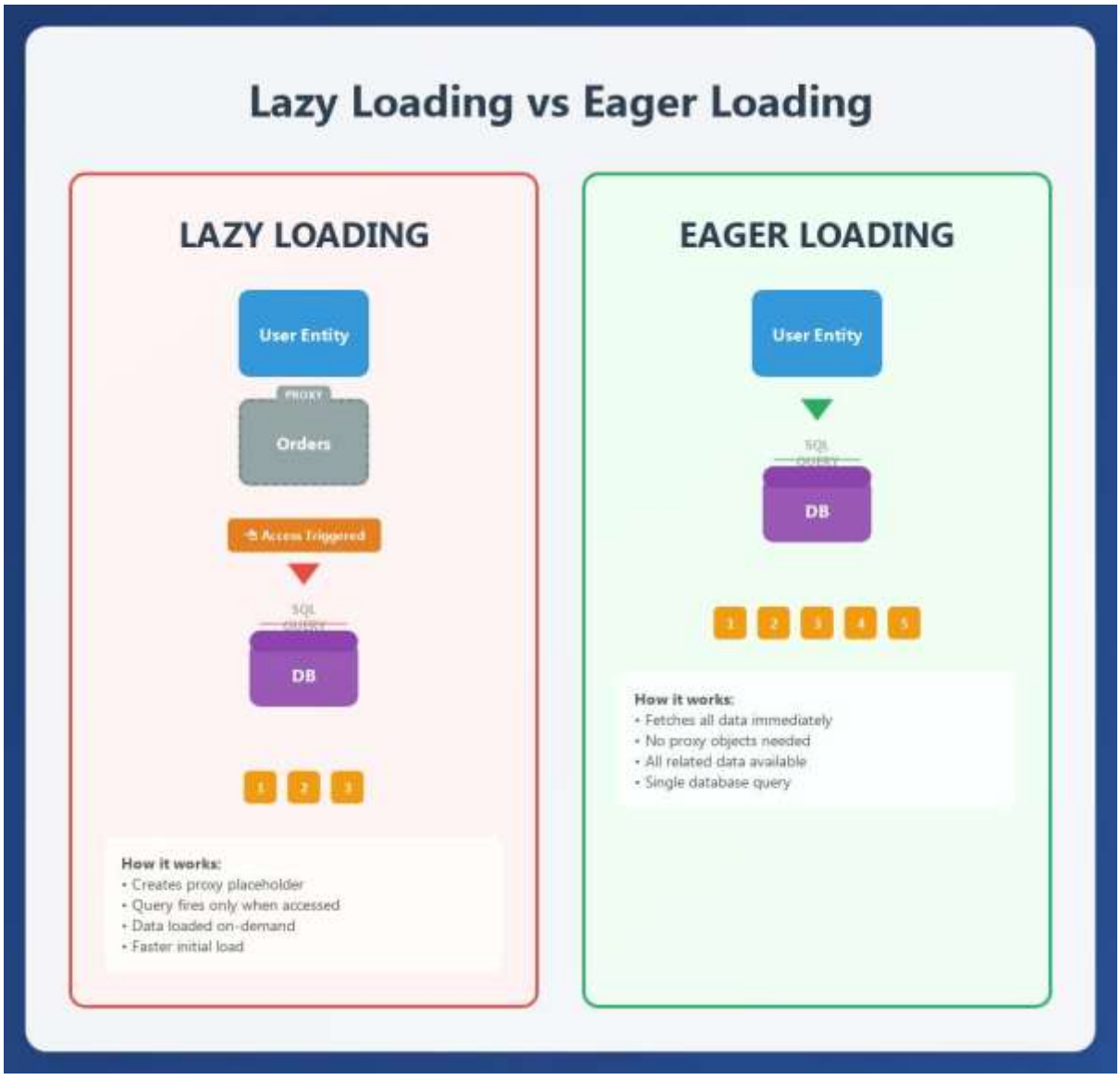
3. Root Cause

The main reason is:

Lazy Loading (Default in JPA)

```
@OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
private List<Employee> employees;
```

- Employees are NOT loaded initially
- When accessed → Hibernate fires separate queries per department



4. Example Domain

We'll use:

- Department (Parent)
- Employee (Child)

5. WITHOUT Fix (N+1 Problem)

Entity Classes

```
@Entity
public class Department {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
    private List<Employee> employees;
}
```

```
@Entity
public class Employee {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

Repository

```
public interface DepartmentRepository extends JpaRepository<Department, Long> {
}
```

Service

```
@Service
public class DepartmentService {

    @Autowired
    private DepartmentRepository repository;

    public List<Department> getAllDepartments() {
        return repository.findAll();
    }
}
```

Controller

```
@RestController
@RequestMapping("/departments")
public class DepartmentController {

    @Autowired
    private DepartmentService service;

    @GetMapping
    public List<Department> getAll() {
        return service.getAllDepartments();
    }
}
```

What Happens Internally?

1. Query 1:

   SELECT * FROM department;

2. For each department:

   SELECT * FROM employee WHERE department_id = ?;

If 5 departments → **6 queries total**

6. WITH Fix (Optimized Solution)

Approach 1: EntityGraph (Recommended in Spring Boot)

Repository

```
public interface DepartmentRepository extends JpaRepository<Department, Long> {

    @EntityGraph(attributePaths = "employees")
    List<Department> findAll();
}
```

What does @EntityGraph do?

- Overrides default lazy loading
- Forces Hibernate to fetch related entities in **single query**
- Internally converts to:

```
SELECT d.*, e.*
FROM department d
LEFT JOIN employee e ON d.id = e.department_id;
```

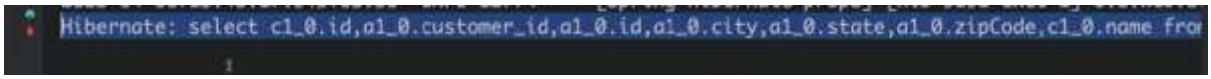
attributePaths

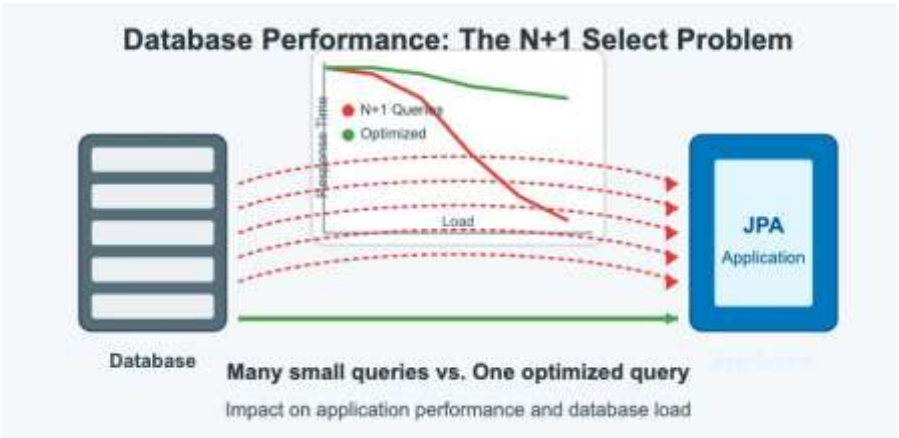
```
@EntityGraph(attributePaths = "employees")
```

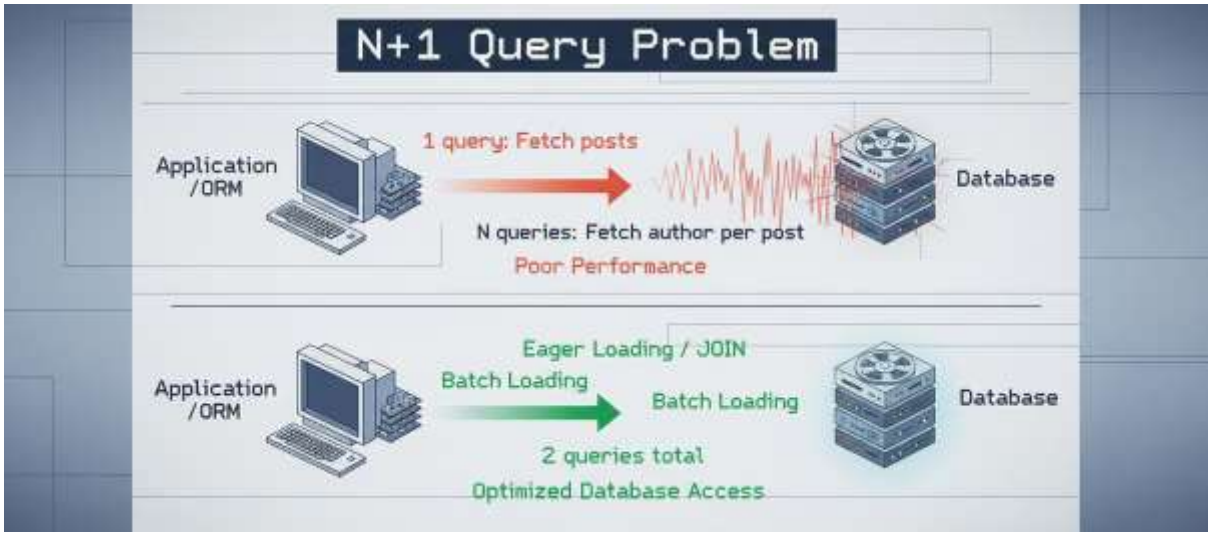
- Tells JPA: "While fetching Department, also fetch employees"

Result

Only **ONE query executed**







Approach 2: JPQL Fetch Join

Repository

```
@Query("SELECT d FROM Department d JOIN FETCH d.employees")
List<Department> findAllWithEmployees();
```

- Explicitly tells Hibernate:
  "Fetch both in one query"

Result SQL

```
SELECT d.*, e.*
FROM department d
INNER JOIN employee e ON d.id = e.department_id;
```

Approach 3: Set FetchType.EAGER

```
@OneToMany(mappedBy = "department", fetch = FetchType.EAGER)
private List<Employee> employees;
```

Why NOT recommended?

- Always loads data (even if not needed)
- Can cause:
  - Memory overhead
  - Cartesian product issues
  - Performance degradation in large systems

Approach 4: Batch Fetching

spring.jpa.properties.hibernate.default_batch_fetch_size=10

How it works

- Groups multiple lazy loads into batches
- Reduces number of queries (not fully 1 query)

Approach 5: DTO Projection

```
@Query("SELECT new com.dto.DepartmentDTO(d.name, e.name) FROM Department d JOIN d.employees e")
List<DepartmentDTO> fetchDTO();
```

Benefit

- Fetch only required fields
- Avoids entity graph overhead

Approach	Queries	Control	Recommended
Default (Lazy)	N+1	Low	No
EntityGraph	1	High	Yes
Fetch Join (JPQL)	1	High	Yes
EAGER Fetch	1	Low	No
Batch Fetch	Reduced	Medium	Sometimes
DTO Projection	1	High	Advanced

8. When Does N+1 Usually Occur?

- OneToMany relationships
- ManyToOne in loops
- Nested entity access inside services/controllers
- REST APIs returning full entity graphs

9. Key Takeaways

- N+1 is a **query explosion problem**
- Root cause = **Lazy loading**
- Best solutions:
    - EntityGraph (Spring Boot way)
    - Fetch Join (JPQL way)
- Avoid:
    - Blind EAGER loading

---

10. Summary

- N+1 Problem = 1 parent query + N child queries
- Caused by Lazy loading
- Leads to performance degradation
- Solved using:
    - EntityGraph
    - Fetch Join
    - DTO projection
- Always monitor SQL logs to detect it

Lazy Loading is causing N+1 Problem

1. What Lazy Loading Actually Means

When you define:

```
@OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
private List<Employee> employees;
```

It means:

“Do NOT load employees when loading Department.

Load them ONLY when they are accessed.”

2. What Hibernate Really Does Internally

Hibernate does NOT put actual data in employees initially.

Instead, it puts a **proxy (placeholder object)**.

Think of it like:

```
Department {
  id = 1,
  name = "IT",
  employees = "Not Loaded Yet (Proxy)"
}
```

3. Step-by-Step Execution (Where Problem Starts)

Step 1: Fetch Departments

```
List<Department> departments = repository.findAll();
```

SQL:

```
SELECT * FROM department;
```

At this point:

- Departments are loaded
- Employees are NOT loaded

Step 2: Access Employees (Trigger Point)

Now somewhere:

```
for (Department d : departments) {
    System.out.println(d.getEmployees().size());
}
```

THIS is the key moment.

4. What Happens Now

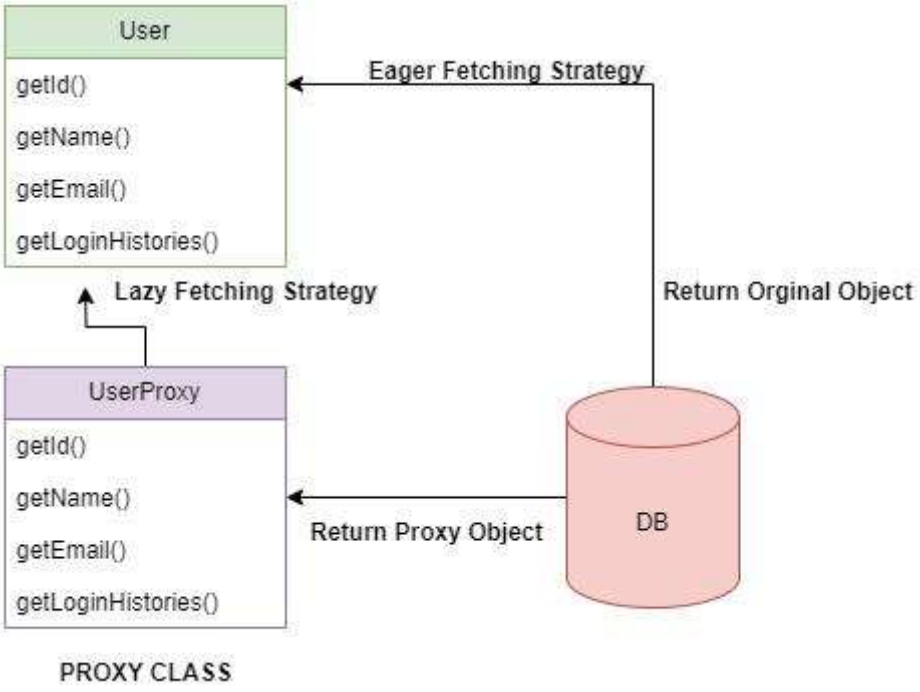
For each department:

Hibernate sees:

“Employees are not loaded yet → I must fetch them”

So it fires:

```
SELECT * FROM employee WHERE department_id = 1;
SELECT * FROM employee WHERE department_id = 2;
SELECT * FROM employee WHERE department_id = 3;
```



6. Why Lazy Loading Causes N+1

Lazy loading itself is NOT bad.

The problem is:

Lazy loading + Iteration = N queries

Because:

- Each access to getEmployees() triggers a query
- And you do it inside a loop

Formula

```
1 query → load departments
+
N queries → load employees per department
=
N+1 problem
```

7. Important Clarification

Lazy loading is NOT always the problem.

It becomes a problem ONLY when:

- You access child data inside a loop
- OR you return entities directly in APIs
- OR you don’t control fetching strategy

8. Real-Life Analogy

Imagine:

- You go to a library and ask for list of books (Departments)
- Librarian gives you only book titles (no content)

Now for each book you say:

- “Give me pages of this book”
- “Now next book…”
- “Now next book…”

You keep going back again and again.

That’s N+1.

9. Why EntityGraph / Fetch Join Fix This

Instead of:

> "Fetch later when needed"

We say:

> "Fetch everything in ONE GO"

So Hibernate does:

```
SELECT d.*, e.*
FROM department d
LEFT JOIN employee e ON d.id = e.department_id;
```

Now:

- No proxy
- No repeated queries
- No N+1

10. Final Summary

- Lazy = "Load later when accessed"
- Access inside loop = "Trigger query every time"
- That repeated triggering = N+1 problem

Lazy loading causes N+1 because related entities are fetched only when accessed, and when accessed inside a loop, Hibernate executes one query per parent entity, leading to N additional queries.

What is Hibernate N+1 Select Problem

- The N + 1 Select problem is a **performance issue** in Hibernate. In this problem, a Java application makes N + 1 database calls (N = number of child objects fetched). For example, if N= 2, the application makes 3 (N+1= 3) database calls.

- Example - Employees and Departments have Many To one relationship . One Department (Parent) can have multiple Employees (Child)

  // Unidirectional mapping . By default its lazy
- Using
  - @OneToMany(fetch = FetchType.LAZY)
  - @JoinColumn(name="Dept_id") // dept_id will be column in Employee table.

- Now we need to fetch all departments .

Ex: One Department can have multiple Employees

Department Entity Has OneToMany relationship with Employee.

And it is LazyLoaded, until it is not used ListOf Employees in Department entity is not fetched

```
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String name;

    @OneToMany(fetch = FetchType.LAZY)
    @JoinColumn(name="Dept_id")
    List<Employee> listOfEmployees;
```

Controller calling all Departments

```
@GetMapping("/dep")
public ResponseEntity<?> fetchAllDepartments(){
    try {
        return new ResponseEntity<List<Department>>(employeeServiceInterface.getAllD
    }catch (Exception e) {
        e.printStackTrace();
        return null;
    }

}
```

Service

```
@Override
public List<Department> getAllDepartmets() {
    List<Department> departments = departmentCrudRepo.findAll();
    //List<Department> departments =   departmentCrudRepo.findWithoutNPlusOne();

    return departments;
}
```

```
1
2 spring.datasource.url= jdbc:mysql://localhost:3306/codedecode?useSSL=false&allowPublicKey
3 spring.datasource.username= root
4 spring.datasource.password= root
5
6 spring.jpa.properties.hibernate.dialect= org.hibernate.dialect.MySQL5Dialect
7 spring.jpa.hibernate.ddl-auto= update
8 spring.jpa.show-sql=true
9 spring.jpa.properties.hibernate.format_sql=true
```

When we hit this controller, then this will call service and it fetches all the departments, and for all departments it fetches all employees → N+1 Queries hit (N+1 Problem)

```
Hibernate:
    select
        d1_0.id,
        d1_0.name
    from
        dept_table d1_0
Hibernate:
    select
        l1_0.dept_id,
        l1_0.id,
        l1_0.name
    from
        employee_table l1_0
    where
        l1_0.dept_id=?
Hibernate:
    select
        l1_0.dept_id,
        l1_0.id,
        l1_0.name
    from
        employee_table l1_0
    where
        l1_0.dept_id=?
Hibernate:
    select
```

What is Hibernate N+1 Select Problem

- What all steps will be done now to fetch all departments -
    - by default its lazy **so first a call goes to Department table and fetch all departments** (only id and name —> No employes list fetched)
    - Then while returning, for each department **now a call goes to fetch list of employees. So N calls goes now, each for 1 department.**

N+1 Problem Solution

What is Hibernate N+1 Select Problem's Solution

- At SQL level, what ORM needs to achieve to avoid N+1 is to fire a query that joins the two tables and get the combined results in single query.
- Spring Data JPA Approach-
    - using left join fetch, we resolve the N+1 problem
    - using attributePaths, Spring Data JPA avoids N+1 problem
- Hibernate Approach
    - Using HQL Query - "from Department d join fetch d.listOfEmployees Employee e"
    - Criteria criteria = session.createCriteria(Department.class); criteria.setFetchMode("listOfEmployees", FetchMode.EAGER);
    -

What is @EntityGraph(attributePaths = {"listOfEmployees"})

- At SQL level, what ORM needs to achieve to avoid N+1 is to fire a query that joins the two tables and get the combined results in single query.
- Spring Data JPA Approach-
    - **using left join fetch,** - we can use JOIN FETCH. The FETCH keyword of the JOIN FETCH statement is JPA-specific. It instructs the persistence provider to not only join the two database tables contained in the query, but also initialize the association on the returned entity. It works with both JOIN and LEFT JOIN statements.
    - **using attributePaths, - EntityGraphs** are introduced in JPA 2.1 and used to allow partial or specified fetching of objects. When an entity has references to other entities we can specify a fetch plan by EntityGraphs in order to determine which fields or properties should be fetched together.

```
10 public interface DepartmentCrudRepo extends JpaRepository<Department, Long> {
11
12     @Query("SELECT p FROM Department p LEFT JOIN FETCH p.listOfEmployees")
13        List<Department> findWithoutNPlusOne();
14
15 //    @EntityGraph(attributePaths = {"listOfEmployees"})
16 //    List<Department> findAll();
17
18
19 }
20
```

```
@Override
public List<Department> getAllDepartmets() {
    //List<Department>  departments =   departmentCrudRepo.findAll();
    List<Department>  departments = departmentCrudRepo.findWithoutNPlusOne();

    return departments;
}
```

Now, only one query executes

```
Hibernate:
    select
        d1_0.id,
        l1_0.dept_id,
        l1_0.id,
        l1_0.name,
        d1_0.name
    from
        dept_table d1_0
    left join
        employee_table l1_0
            on d1_0.id=l1_0.dept_id
```

Spring Data JPA – Approach using Entity Graph

```
public interface DepartmentCrudRepo extends JpaRepository<Department, Long> {

    @Query("SELECT p FROM Department p LEFT JOIN FETCH p.listOfEmployees")
       List<Department> findWithoutNPlusOne();

    @EntityGraph(attributePaths = {"listOfEmployees"})
    List<Department> findAll();
```

- Hibernate Approach
    - **Using HQL Query** - "from Department d join fetch d.listOfEmployees Employee e'
    - **Using Criteria**
        - Criteria criteria = session.createCriteria(Department.class);
          criteria.setFetchMode("listOfEmployees", FetchMode.EAGER);